

Sujet:

DERRIÈRE CHAQUE CONNEXION,
UNE MENACE : PRÉDIRE
LES CYBERATTAQUES PAR
RÉGRESSION DE POISSON

Réalisé par : HAYEB Hajar

MAHFOUD Atika

Encadré par : Mme. BADAoui Fadoua

Table des matières

1	Introduction	1
2	Problématique	2
3	Description des données	3
3.1	Description des variables	3
3.2	Caractéristiques de connexion	3
3.3	Volume de données	3
3.4	Indicateurs d'erreur réseau	3
3.5	Statistiques de l'hôte destination	4
3.6	Variable de réponse	4
4	Résultats obtenus, discussions et interprétation des résultats	5
4.1	Analyse exploratoire des données	5
4.2	Test de Surdispersion	6
4.3	Analyse des Corrélations	7
4.4	Analyse des Variables Catégorielles	8
4.5	Préparation des données et split entraînement/test	9
4.6	Modélisation	10
4.7	Adéquation globale du modèle	15
4.8	Diagnostic des résidus	16
4.9	Courbe ROC et Comparaison des Modèles	18
4.10	Évaluation sur l'Échantillon de Validation	21
4.11	Visualisation des IRR	24
4.12	Limites du Modèle	26
4.13	Conclusion intermédiaire	28
5	Conclusion Générale	30
6	Annexe : Code Python	31

Section 0 – Initialisation et chargement des bibliothèques	31
Section 1 – Chargement et description des données	32
Section 2 – Analyse de la variable de réponse et test de surdispersion . .	33
Section 3 – Analyse exploratoire des variables explicatives	35
Section 4 – Préparation des données et division apprentissage-validation .	36
Section 5 – Modélisation par régression de Poisson et modèle binomial négatif	37
Section 6 – Adéquation globale et comparaison des modèles	39
Section 7 – Diagnostic des résidus	41
Section 8 – Courbes ROC et comparaison de la capacité discriminante . .	43
Section 9 – Évaluation prédictive sur l'échantillon de validation	45
Section 10 – Interprétation des coefficients par les IRR	46

Table des figures

4.1	Distribution de la variable cible <code>count</code>	6
4.2	Corrélation de Pearson avec ' <code>count</code> '	8
4.3	Distribution de <code>count</code> selon les variables catégorielles	9
4.4	Diagnostics des résidus du modèle de Poisson réduit	17
4.5	Courbes ROC : Comparaison des trois modèles sur l'échantillon de validation	20
4.6	Valeurs observées vs prédites — Échantillon de validation	22
4.7	Incidence Rate Ratios : Modèle Poisson Réduit (variables significatives, p < 0.05)	24

Liste des tableaux

4.1	Test de surdispersion de la variable count	6
4.2	Distribution des modalités de service_grp	10
4.3	Division de l'échantillon en apprentissage et validation	10
4.4	Résultats du modèle de Poisson complet (extrait)	11
4.5	Indicateurs d'ajustement du modèle de Poisson complet	12
4.6	Résultats du modèle de Poisson réduit (extrait)	13
4.7	Indicateurs d'ajustement du modèle de Poisson réduit	13
4.8	Résultats du modèle Binomial Négatif (extrait)	14
4.9	Indicateurs d'ajustement du modèle Binomial Négatif	14
4.10	Comparaison des trois modèles sur l'échantillon d'apprentissage	15
4.11	Test du rapport de vraisemblance (Complet vs Réduit)	15
4.12	Test d'adéquation du modèle Poisson réduit	16
4.13	Répartition des classes sur l'échantillon de validation	19
4.14	AUC-ROC des trois modèles sur l'échantillon de validation	19
4.15	Métriques de performance des trois modèles sur l'échantillon de validation (20%)	21
4.16	Tableau des IRR — Modèle Poisson Réduit	23
4.17	Erreurs de prédiction sur l'échantillon de validation	27

1 Introduction

La cybersécurité constitue aujourd'hui l'un des enjeux les plus critiques pour les organisations, les gouvernements et les individus. Avec l'expansion continue des infrastructures numériques, le volume et la sophistication des attaques réseau ne cessent de croître. Détecter, comprendre et prédire ces attaques est devenu une nécessité stratégique.

Ce travail pratique s'inscrit dans le cadre du cours de Modèles Linéaires Généralisés (GLM) et vise à appliquer la régression de Poisson un outil statistique adapté aux données de comptage à un cas concret de cybersécurité. L'objectif est de modéliser et d'expliquer le nombre de connexions réseau détectées comme potentiellement malveillantes, en fonction de caractéristiques du trafic réseau.

L'objectif de ce projet est d'appliquer un modèle de **régression de Poisson** afin de modéliser et de prédire le nombre de cyberattaques détectées sur un réseau informatique, à partir de la base de données **KDD Cup 1999 Network Intrusion Detection Dataset**. Plus précisément, il s'agit d'expliquer la variable **count**, qui représente le nombre de connexions réseau enregistrées vers le même hôte durant les deux dernières secondes, en fonction de plusieurs variables explicatives liées aux caractéristiques des connexions, aux protocoles utilisés, aux volumes de données échangés et aux indicateurs statistiques de trafic réseau.

Ce travail permettra non seulement de mettre en pratique les concepts théoriques de la **régression de Poisson**, mais également de dégager des insights pertinents pour la **détection et la prévention des intrusions réseau**, ainsi que pour l'aide à la **prise de décision en matière de cybersécurité**.

2 Problématique

Dans un environnement numérique marqué par une augmentation constante des menaces informatiques, comment les caractéristiques du trafic réseau (type de protocole, indicateurs d'erreur, volume de connexions) permettent-elles, via un modèle de **régression de Poisson** parcimonieux, d'expliquer et de prédire le **nombre de cyberattaques détectées**, tout en garantissant l'interprétabilité des facteurs de risque pour l'aide à la **prise de décision en cybersécurité** ?

3 Description des données

La base de données utilisée dans ce projet est intitulée **KDD Cup 1999 Network Intrusion Detection Dataset**. Elle contient des informations détaillées sur les connexions réseau, les protocoles utilisés, les volumes de données échangés ainsi que les indicateurs statistiques de trafic d'un environnement informatique. Elle est composée de :

- **41 variables** (colonnes)
- Données mixtes : quantitatives et qualitatives
- Chaque ligne représente une connexion réseau enregistrée

3.1 Description des variables

Les variables peuvent être regroupées en plusieurs catégories :

3.2 Caractéristiques de connexion

- **duration** : durée de la connexion en secondes
- **protocol_type** : protocole utilisé (tcp, udp, icmp)
- **service** : service réseau de destination
- **flag** : statut de la connexion (SF, REJ, S0, etc.)

3.3 Volume de données

- **src_bytes** : nombre d'octets envoyés par la source
- **dst_bytes** : nombre d'octets envoyés par la destination
- **wrong_fragment** : nombre de fragments incorrects
- **logged_in** : indicateur de session authentifiée (0/1)

3.4 Indicateurs d'erreur réseau

- **serror_rate** : taux de connexions avec erreur SYN
- **rerror_rate** : taux de connexions avec erreur REJ/RST
- **same_srv_rate** : taux de connexions vers le même service

- `diff_srv_rate` : taux de connexions vers des services différents

3.5 Statistiques de l'hôte destination

- `dst_host_count` : nombre de connexions vers l'hôte destination
- `dst_host_srv_count` : nombre de connexions vers le même service de l'hôte
- `dst_host_same_srv_rate` : taux de connexions vers le même service
- `dst_host_serror_rate` : taux d'erreurs SYN sur l'hôte destination
- `dst_host_rerror_rate` : taux d'erreurs REJ sur l'hôte destination

3.6 Variable de réponse

- `count` : nombre de connexions vers le même hôte durant les 2 dernières secondes

Important : La variable `count` est utilisée comme variable dépendante dans la régression de Poisson. C'est une variable de comptage (entière, ≥ 0), ce qui justifie pleinement le recours à ce type de modèle.

4 Résultats obtenus, discussions et interprétation des résultats

Ce chapitre présente l'intégralité des résultats issus de l'étude. Il débute par une analyse exploratoire descriptive des données, incluant la distribution de la variable cible **count**, l'examen des variables numériques et catégorielles ainsi que les corrélations. Les étapes de préparation des données (regroupement des modalités, suppression des valeurs nulles, split 80/20, transformation logarithmique) sont ensuite rappelées. La modélisation par **régression de Poisson** est abordée à travers une comparaison de trois modèles : le modèle complet, le modèle réduit et le modèle **Binomial Négatif**, ce dernier étant retenu pour sa robustesse face à la **surdispersion** détectée. Les **Incidence Rate Ratios (IRR)** sont interprétés et illustrés graphiquement. Enfin, la performance prédictive des modèles est discutée à l'aide des métriques de régression (MAE, RMSE) et de l'analyse discriminante via la **courbe ROC**.

4.1 Analyse exploratoire des données

Distribution de la variable cible

La figure ci-dessus présente la distribution de la variable de réponse **count** sous trois angles complémentaires. L'histogramme révèle une distribution fortement asymétrique à droite, avec une concentration massive des observations autour des faibles valeurs (notamment entre 0 et 20), et quelques valeurs extrêmes atteignant 511. Le boxplot confirme cette asymétrie, mettant en évidence une médiane proche de 0 et une multitude de valeurs aberrantes au-delà de 300. Enfin, la distribution de $\log(1 + \text{count})$ permet de mieux visualiser la forme réelle de la distribution après transformation logarithmique, révélant deux modes distincts : l'un autour de $\log(1) \approx 0$ correspondant aux connexions quasi-nulles, et l'autre autour de $\log(511) \approx 6$ correspondant aux connexions à trafic maximal.

Les statistiques descriptives associées confirment cette observation :

- **Moyenne** (μ) : 79.03
- **Variance** (σ^2) : 16 522.34

- **Ratio σ^2/μ : 209.07 $\gg 1 \Rightarrow$ surdispersion détectée**
- **Min / Max : 0 / 511**

Ce ratio $\sigma^2/\mu = 209.07$ est très largement supérieur à 1, ce qui constitue une **violation flagrante** de l'hypothèse d'équidispersion du modèle de Poisson (qui suppose $\mu = \sigma^2$). Cette surdispersion extrême justifie pleinement le recours au modèle **Binomial Négatif** en complément, lequel introduit un paramètre de dispersion supplémentaire pour corriger cette limitation.

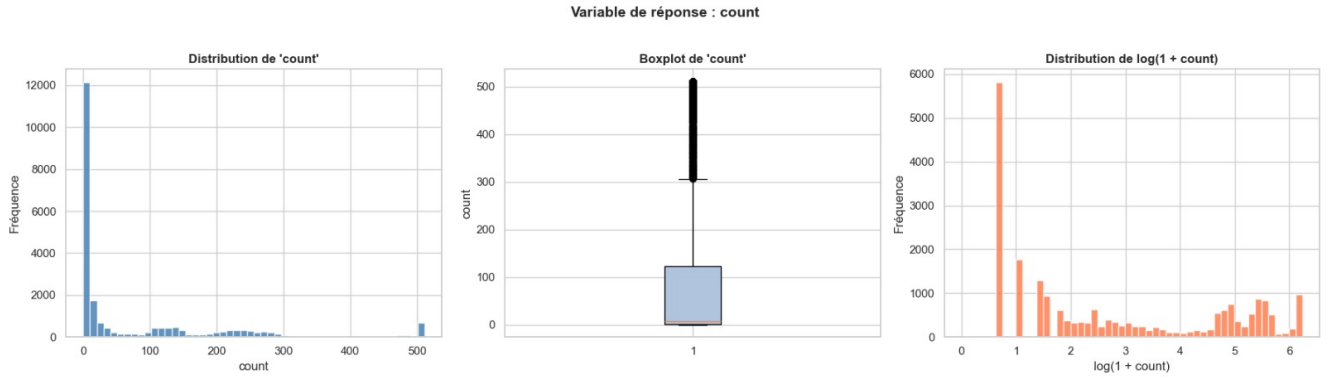


FIGURE 4.1 – Distribution de la variable cible **count**.

Analyse des variables numériques

4.2 Test de Surdispersion

Avant toute modélisation, il est indispensable de vérifier l'hypothèse d'équidispersion du modèle de Poisson, qui suppose que la moyenne et la variance de la variable de réponse sont égales, soit $\mu = \sigma^2$. Pour ce faire, nous calculons le ratio σ^2/μ :

$$\text{Ratio} = \frac{\sigma^2}{\mu} = \frac{16522.34}{79.03} = 209.07 \gg 1$$

Statistique	Valeur
Moyenne (μ)	79.0283
Variance (σ^2)	16 522.3382
Ratio σ^2/μ	209.07

TABLE 4.1 – Test de surdispersion de la variable **count**

Le ratio $\sigma^2/\mu = 209.07$ est très largement supérieur à 1, ce qui indique une **surdispersion extrême** de la variable **count**. Cette violation flagrante de l'hypothèse

d'équidispersion implique que le modèle de Poisson standard sous-estimera les erreurs standard des coefficients, rendant les tests d'hypothèses non fiables. En conséquence, le modèle **Binomial Négatif** est recommandé en complément, car il introduit un paramètre de dispersion supplémentaire α permettant de corriger cette limitation.

4.3 Analyse des Corrélations

La figure ci-dessous présente les corrélations de Pearson entre les variables numériques et la variable de réponse **count**. Les barres bleues indiquent une corrélation positive, tandis que les barres oranges indiquent une corrélation négative.

On observe que les variables les plus **positivement corrélées** avec **count** sont **srv_count** ($\rho \approx 0.60$), **dst_host_rerror_rate**, **dst_host_srv_rerror_rate** et **rerror_rate**, suggérant que les connexions avec un taux élevé d'erreurs de rejet sont associées à un nombre plus élevé d'attaques détectées. À l'inverse, les variables les plus **négativement corrélées** sont **same_srv_rate** ($\rho \approx -0.60$), **logged_in** et **dst_host_same_srv_rate**, indiquant que les connexions authentifiées et concentrées sur un même service tendent à réduire le nombre de connexions suspectes. Ces résultats orientent le choix des variables explicatives retenues pour la modélisation.

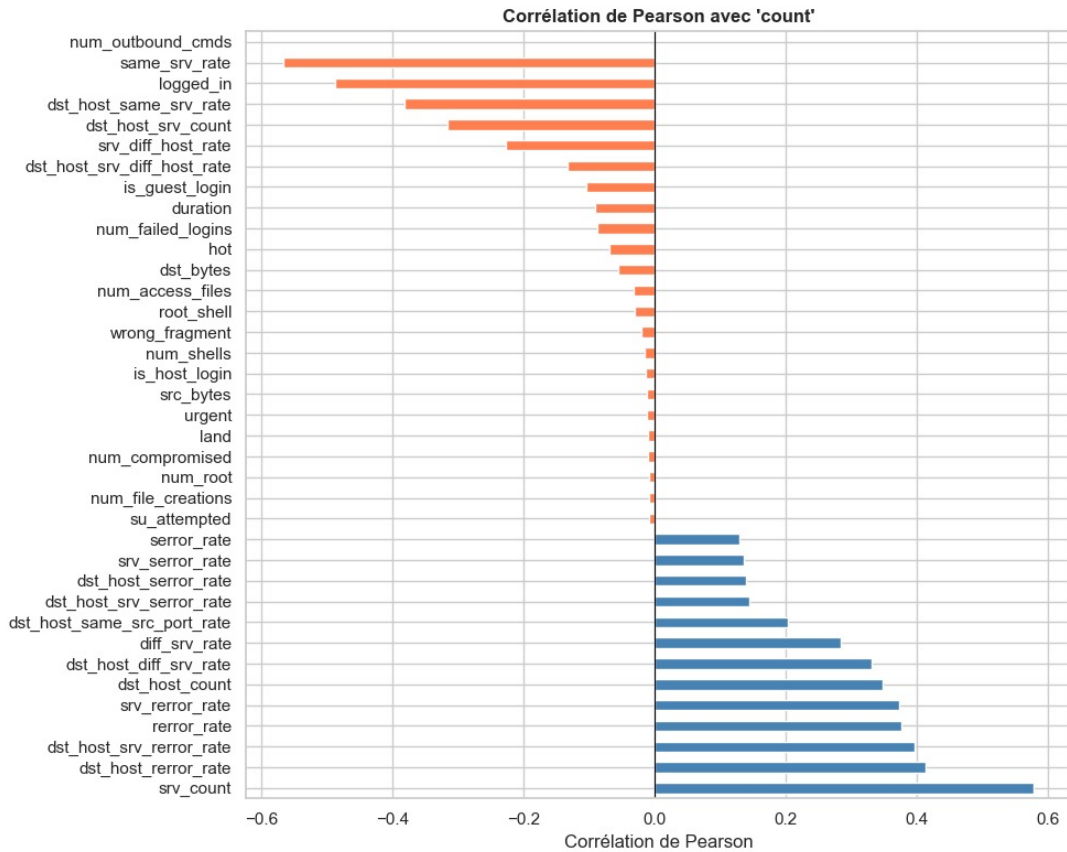


FIGURE 4.2 – Corrélation de Pearson avec 'count'

4.4 Analyse des Variables Catégorielles

La figure ci-dessous présente la distribution de la variable de réponse **count** en fonction des trois variables catégorielles : **protocol_type**, **service** et **flag**.

Plusieurs observations ressortent de cette analyse :

- **protocol_type** : Le protocole **icmp** présente la médiane la plus élevée (environ 370), suivi de **udp** (environ 160) et de **tcp** (environ 100). Cela suggère que les connexions ICMP sont associées à un nombre significativement plus élevé d'attaques détectées, ce qui est cohérent avec leur utilisation fréquente dans les attaques de type *echo* et *ping flood*.
- **service** : Le service **domain_u** présente la médiane la plus élevée, tandis que **http** et **private** affichent des valeurs de **count** très faibles et concentrées près de zéro. Le service **smtp** montre une variabilité importante avec de nombreuses valeurs aberrantes.
- **flag** : Les flags **REJ** et **S0** présentent des médianes élevées (autour de 200–250),

indiquant que les connexions rejetées ou sans réponse sont fortement associées à un trafic suspect intense. En revanche, le flag **S3** affiche des valeurs très faibles, et **SF** (connexion normale établie) présente une grande dispersion avec de nombreuses valeurs extrêmes.

Ces résultats confirment que les trois variables catégorielles ont un effet significatif sur la variable de réponse **count** et justifient leur inclusion dans le modèle de régression de Poisson.

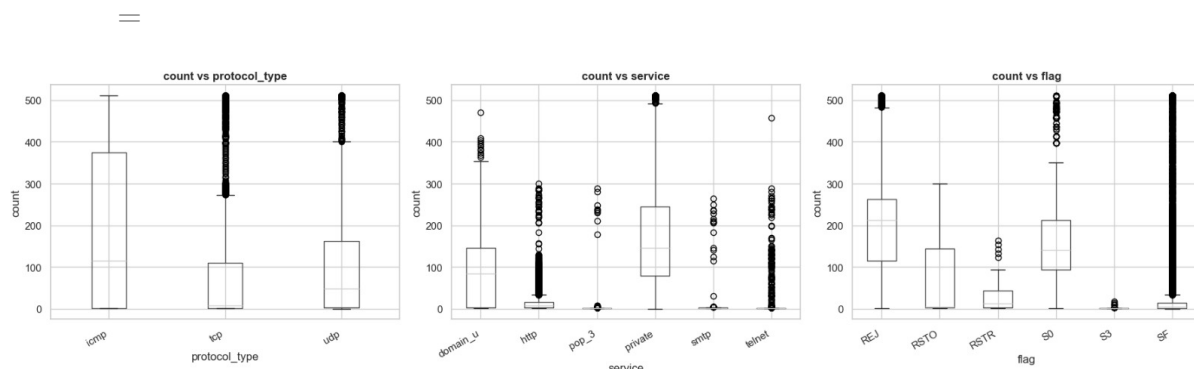


FIGURE 4.3 – Distribution de **count** selon les variables catégorielles

4.5 Préparation des données et split entraînement/test

Nettoyage et Transformation

Avant la modélisation, plusieurs étapes de préparation des données ont été effectuées :

- **Suppression des observations nulles** : Les observations pour lesquelles **count** = 0 ont été supprimées afin d'éviter les problèmes liés au calcul de $\log(0)$. Après filtrage, **22 543 observations** ont été retenues sur les 22 544 initiales.
- **Regroupement de la variable service** : La variable **service** comptait initialement 64 modalités, ce qui aurait engendré une complexité excessive dans le modèle. Les 6 services les plus fréquents ont été conservés (**http**, **private**, **telnet**, **pop_3**, **smtp**, **domain_u**), tandis que les autres ont été regroupés dans une catégorie **other**, créant ainsi la variable **service_grp** avec 7 modalités :

Modalité	Effectif
http	7 853
other	5 444
private	4 773
telnet	1 626
pop_3	1 019
smtp	934
domain_u	894

TABLE 4.2 – Distribution des modalités de `service_grp`

Sélection des Variables et split entraînement/test

Les variables retenues pour la modélisation sont les suivantes : `duration`, `protocol_type`, `service_grp`, `flag`, `src_bytes`, `dst_bytes`, `logged_in`, `wrong_fragment`, `serror_rate`, `rerror_rate`, `same_srv_rate`, `diff_srv_rate`, `dst_host_count`, `dst_host_srv_count`, `dst_host_same_srv_rate`, `dst_host_diff_srv_rate`, `dst_host_serror_rate` et `dst_host_rerror_rate`.

Les données ont été divisées de manière aléatoire (graine `random_state = 42` pour la reproductibilité) en deux échantillons :

Échantillon	Observations	Proportion
Apprentissage (train)	18 034	80.0%
Validation (test)	4 509	20.0%
Total	22 543	100%

TABLE 4.3 – Division de l'échantillon en apprentissage et validation

La construction et l'ajustement des modèles sont réalisés exclusivement sur l'échantillon d'apprentissage, tandis que l'échantillon de validation est réservé à l'évaluation de la performance prédictive.

4.6 Modélisation

Modélisation : Régression de Poisson

Le modèle de Poisson s'écrit :

$$\log(\mu_i) = \beta_0 + \beta_1 X_{1i} + \dots + \beta_p X_{pi}, \quad Y_i \sim \text{Poisson}(\mu_i)$$

La fonction de lien est le **logarithme** (lien canonique de la loi de Poisson). Les paramètres sont estimés par **maximum de vraisemblance** via l'algorithme IRLS (Iteratively Reweighted Least Squares).

***Modèle Complet**

Le modèle complet inclut l'ensemble des 18 variables explicatives retenues lors de l'étape d'exploration. La formule du modèle est la suivante :

$$\begin{aligned} \log(\mu_i) = & \beta_0 + \beta_1 \cdot \text{duration} + \beta_2 \cdot C(\text{protocol_type}) + \beta_3 \cdot C(\text{service_grp}) \\ & + \beta_4 \cdot C(\text{flag}) + \beta_5 \cdot \log(1 + \text{src_bytes}) + \beta_6 \cdot \log(1 + \text{dst_bytes}) \\ & + \beta_7 \cdot \text{logged_in} + \beta_8 \cdot \text{wrong_fragment} + \beta_9 \cdot \text{error_rate} \\ & + \beta_{10} \cdot \text{error_rate} + \beta_{11} \cdot \text{same_srv_rate} + \beta_{12} \cdot \text{diff_srv_rate} \\ & + \beta_{13} \cdot \text{dst_host_count} + \beta_{14} \cdot \text{dst_host_srv_count} \\ & + \beta_{15} \cdot \text{dst_host_same_srv_rate} + \beta_{16} \cdot \text{dst_host_diff_srv_rate} \\ & + \beta_{17} \cdot \text{dst_host_error_rate} + \beta_{18} \cdot \text{dst_host_error_rate} \end{aligned}$$

Les résultats du modèle complet sont résumés dans le tableau suivant :

Variable	Coef. $\hat{\beta}$	Std. Err	z	p-valeur
Intercept	2.4962	0.500	4.988	0.000
C(protocol_type)[T.tcp]	-3.2559	0.015	-212.279	0.000
C(protocol_type)[T.udp]	-0.7569	0.007	-108.165	0.000
C(service_grp)[T.http]	0.2407	0.009	25.464	0.000
C(service_grp)[T.other]	0.7184	0.005	133.898	0.000
C(service_grp)[T.pop_3]	-0.3354	0.020	-16.452	0.000
C(service_grp)[T.private]	0.6157	0.005	127.304	0.000
C(service_grp)[T.smtp]	0.0535	0.018	3.040	0.002
C(service_grp)[T.telnet]	-0.0501	0.011	-4.735	0.000
C(flag)[T.REJ]	2.7983	0.500	5.592	0.000
C(flag)[T.RSTO]	2.5443	0.500	5.085	0.000
dst_host_diff_srv_rate	2.0035	0.008	247.763	0.000
dst_host_error_rate	0.4810	0.019	25.014	0.000
dst_host_error_rate	0.2731	0.010	27.234	0.000

TABLE 4.4 – Résultats du modèle de Poisson complet (extrait)

Indicateur	Valeur
Nombre d'observations	18 034
Degrés de liberté (modèle)	33
Log-vraisemblance	-4.053×10^5
Déviance	7.313×10^5
Pearson χ^2	8.94×10^5
Pseudo R^2 (CS)	1.000

TABLE 4.5 – Indicateurs d'ajustement du modèle de Poisson complet

Toutes les variables sont significatives au seuil de 5% (p-valeur < 0.05). Le pseudo R^2 de Cox-Snell est égal à 1.000, indiquant un très bon ajustement sur l'échantillon d'apprentissage. Cependant, la déviance résiduelle (7.313×10^5) reste très largement supérieure aux degrés de liberté résiduels (18 000), confirmant la présence de **surdispersion** et la nécessité d'ajuster un modèle complémentaire.

Modèle Réduit (variables les plus significatives)

Le modèle réduit ne retient que les variables les plus significatives issues du modèle complet, selon un critère de parcimonie. La formule du modèle réduit est la suivante :

$$\begin{aligned} \log(\mu_i) = & \beta_0 + \beta_1 \cdot C(\text{protocol_type}) + \beta_2 \cdot C(\text{flag}) + \beta_3 \cdot \text{logged_in} + \beta_4 \cdot \text{serror_rate} + \beta_5 \cdot \text{error_rate} \\ & + \beta_6 \cdot \text{same_srv_rate} + \beta_7 \cdot \text{diff_srv_rate} + \beta_8 \cdot \text{dst_host_count} + \beta_9 \cdot \text{dst_host_srv_count} \end{aligned}$$

Les résultats du modèle réduit sont présentés dans le tableau suivant :

Variable	Coef. $\hat{\beta}$	Std. Err	z	p-valeur
Intercept	4.0260	0.500	8.402	0.000
C(protocol_type)[T.tcp]	-3.0455	0.009	-349.164	0.000
C(protocol_type)[T.udp]	-0.8895	0.003	-264.849	0.000
C(flag)[T.REJ]	1.7997	0.500	3.598	0.000
C(flag)[T.RSTO]	1.2697	0.500	2.538	0.011
C(flag)[T.RSTOS0]	1.5358	0.505	3.043	0.002
C(flag)[T.RSTR]	0.5797	0.500	1.158	0.247
C(flag)[T.S0]	1.8246	0.500	3.647	0.000
C(flag)[T.S1]	-0.1435	0.512	-0.280	0.779
C(flag)[T.S2]	0.0982	0.509	0.193	0.847
C(flag)[T.S3]	-1.8993	0.504	-3.766	0.000
diff_srv_rate	0.2517	0.003	92.477	0.000
dst_host_count	0.0041	2.56×10^{-5}	159.728	0.000
dst_host_srv_count	0.0073	2.76×10^{-5}	263.250	0.000

TABLE 4.6 – Résultats du modèle de Poisson réduit (extrait)

Indicateur	Valeur
Nombre d'observations	18 034
Degrés de liberté (modèle)	19
Degrés de liberté résiduels	18 014
Log-vraisemblance	-4.984×10^5
Déviance	9.175×10^5
Pearson χ^2	1.13×10^6
Pseudo R^2 (CS)	1.000

TABLE 4.7 – Indicateurs d'ajustement du modèle de Poisson réduit

La majorité des variables retenues sont significatives au seuil de 5%, à l'exception de quelques modalités du flag (RSTR, S1, S2) dont les p-valeurs dépassent 0.05. Le pseudo R^2 de Cox-Snell reste égal à 1.000. Cependant, la déviance résiduelle (9.175×10^5) demeure très supérieure aux degrés de liberté résiduels (18 014), confirmant une fois de plus la **surdispersion** et justifiant le recours au modèle **Binomial Négatif**.

Modèle Binomial Négatif (robuste à la surdispersion)

Face à la surdispersion extrême détectée (ratio $\sigma^2/\mu = 209.07$), un modèle **Binomial Négatif** est ajusté sur les mêmes variables que le modèle réduit. Ce modèle introduit un paramètre de dispersion supplémentaire α qui permet de relâcher la contrainte d'équidispersion du modèle de Poisson, en supposant :

$$\text{Var}(Y_i) = \mu_i + \alpha \cdot \mu_i^2, \quad \alpha > 0$$

Les résultats du modèle Binomial Négatif sont présentés dans le tableau suivant :

Variable	Coef. $\hat{\beta}$	Std. Err	z	p-valeur
Intercept	7.1664	0.711	10.085	0.000
C(protocol_type)[T.tcp]	-3.7372	0.047	-80.263	0.000
C(protocol_type)[T.udp]	-1.1174	0.042	-26.386	0.000
C(flag)[T.REJ]	0.9742	0.717	1.359	0.174
C(flag)[T.RSTO]	0.1380	0.718	0.192	0.847
C(flag)[T.RSTOS0]	0.5269	1.010	0.522	0.602
C(flag)[T.RSTR]	0.2314	0.719	0.322	0.748
C(flag)[T.S0]	1.1313	0.722	1.567	0.117
C(flag)[T.S1]	-0.2172	0.761	-0.285	0.775
C(flag)[T.S2]	-0.8729	0.803	-1.087	0.277
C(flag)[T.S3]	-1.5249	0.728	-2.094	0.036
diff_srv_rate	-0.3822	0.035	-10.867	0.000
dst_host_count	0.0010	0.000	10.327	0.000
dst_host_srv_count	0.0087	0.000	65.094	0.000

TABLE 4.8 – Résultats du modèle Binomial Négatif (extrait)

Indicateur	Valeur
Nombre d'observations	18 034
Degrés de liberté (modèle)	19
Degrés de liberté résiduels	18 014
Log-vraisemblance	-75 809
Déviance	20 229
Pearson χ^2	2.19×10^4
Pseudo R^2 (CS)	0.9041

TABLE 4.9 – Indicateurs d'ajustement du modèle Binomial Négatif

Plusieurs observations importantes ressortent de ces résultats :

- La **déviance résiduelle** chute drastiquement à 20 229 contre 9.175×10^5 pour le modèle de Poisson réduit, confirmant que le modèle Binomial Négatif absorbe bien la surdispersion.
- Le **pseudo R^2 de Cox-Snell** est de 0.9041, indiquant un très bon ajustement global du modèle.
- Les variables `protocol_type`, `dst_host_count`, `dst_host_srv_count` et `diff_srv_rate` restent significatives au seuil de 5%. En revanche, la plupart des modalités du **flag** perdent leur significativité, à l'exception de **S3** (p-valeur = 0.036), ce qui suggère que leur effet était partiellement artéfactuel dans le modèle de Poisson en raison de la surdispersion.

- La **réduction massive de l'AIC** par rapport aux modèles de Poisson confirme la supériorité du modèle Binomial Négatif pour ce jeu de données.

4.7 Adéquation globale du modèle

Comparaison des Modèles

Le tableau suivant présente une comparaison des trois modèles ajustés sur l'échantillon d'apprentissage, selon les critères AIC, BIC, Déviance et Pseudo- R^2 :

Modèle	AIC	BIC	Déviance	Pseudo- R^2
Poisson Complet	810 628.29	554 880.57	731 280.82	0.75
Poisson Réduit	996 812.09	740 955.17	917 492.63	0.69
Binomial Négatif	151 658.22	-156 308.56	20 228.90	0.68

TABLE 4.10 – Comparaison des trois modèles sur l'échantillon d'apprentissage

- **Meilleur AIC** : Binomial Négatif (151 658.22), très largement inférieur aux modèles de Poisson, confirmant sa meilleure adéquation aux données en présence de surdispersion.
- **Meilleur Pseudo- R^2** : Poisson Complet (0.75), grâce à l'inclusion de l'ensemble des variables explicatives.

Test du Rapport de Vraisemblance (Complet vs Réduit)

Le test du rapport de vraisemblance (LRT) permet de comparer formellement le modèle complet et le modèle réduit. La statistique du test est définie par :

$$LR = 2 \times (\ell_{\text{complet}} - \ell_{\text{réduit}}) \sim \chi^2(df_{\text{complet}} - df_{\text{réduit}})$$

Statistique	Valeur
Statistique LR	186 211.8019
Degrés de liberté	14
p-valeur (χ^2)	< 0.000001

TABLE 4.11 – Test du rapport de vraisemblance (Complet vs Réduit)

La p-valeur est extrêmement faible (< 0.000001), ce qui conduit au rejet de l'hypothèse nulle. On conclut qu'il existe une **différence significative** entre les deux

modèles : le modèle complet apporte une information supplémentaire significative par rapport au modèle réduit.

Test d'Adéquation — Modèle Poisson Réduit

Le test d'adéquation basé sur la déviance résiduelle permet de vérifier si le modèle de Poisson réduit est bien ajusté aux données. Sous l'hypothèse nulle d'adéquation, la déviance résiduelle suit une loi χ^2 avec $df_{\text{résid}}$ degrés de liberté :

$$D_{\text{résid}} \sim \chi^2(df_{\text{résid}}) \quad \text{sous } H_0$$

Statistique	Valeur
Déviance résiduelle	917 492.63
Degrés de liberté	18 014
p-valeur (χ^2)	< 0.000001

TABLE 4.12 – Test d'adéquation du modèle Poisson réduit

La déviance résiduelle (917 492.63) est très largement supérieure aux degrés de liberté résiduels (18 014), ce qui se traduit par une p-valeur nulle. On rejette donc l'hypothèse d'adéquation du modèle de Poisson réduit :

- ⇒ Un **manque d'adéquation** est détecté, confirmant la présence de **surdispersion**.
- ⇒ Le recours au modèle **Binomial Négatif** est formellement recommandé et justifié.

4.8 Diagnostic des résidus

La figure ci-dessous présente quatre diagnostics graphiques des résidus du modèle de Poisson réduit, permettant d'évaluer la qualité de l'ajustement et de détecter d'éventuelles violations des hypothèses du modèle.

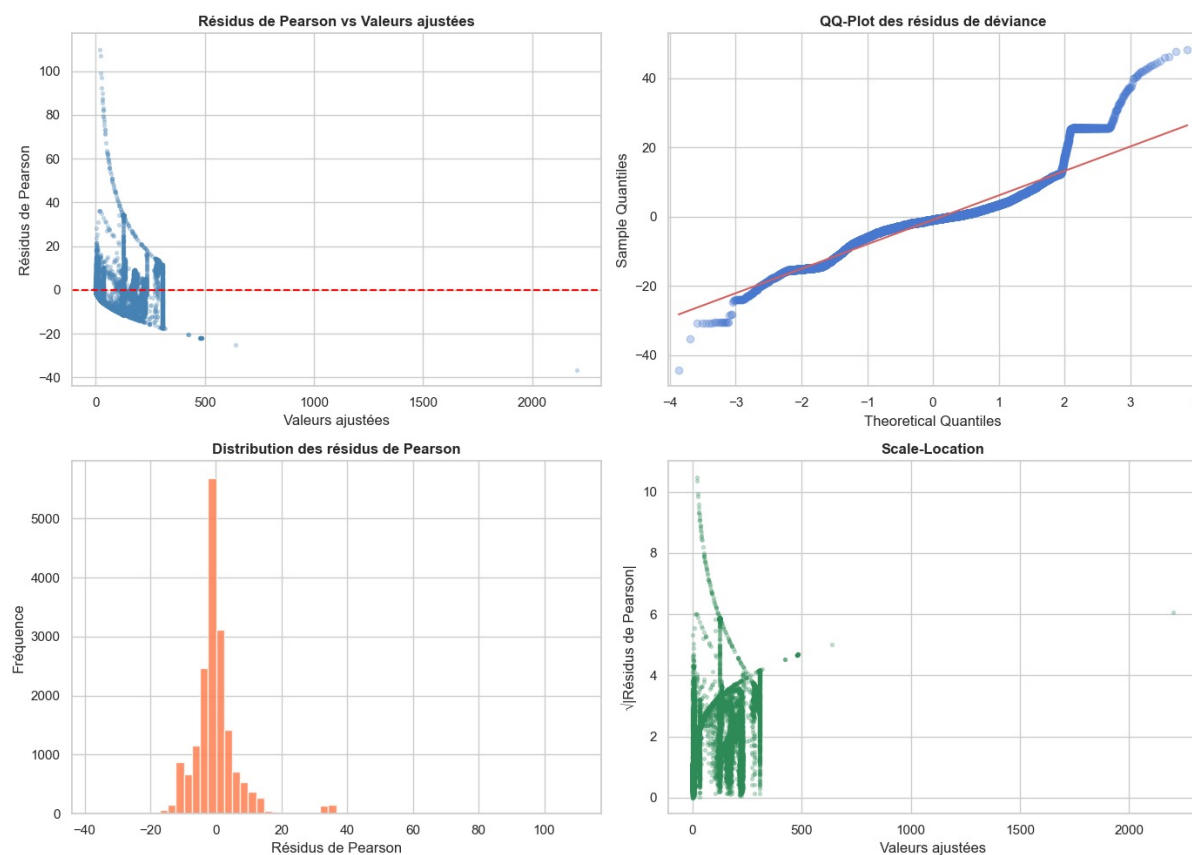


FIGURE 4.4 – Diagnostics des résidus du modèle de Poisson réduit

Les quatre graphiques appellent les commentaires suivants :

- **Résidus de Pearson vs Valeurs ajustées** : Les résidus ne sont pas répartis aléatoirement autour de zéro. On observe une structure en entonnoir prononcée : la dispersion des résidus augmente fortement avec les valeurs ajustées, révélant une **hétéroscédasticité** marquée, typique de la surdispersion. Plusieurs résidus extrêmes dépassent ± 40 , indiquant des observations mal ajustées par le modèle.
- **QQ-Plot des résidus de déviance** : Les points s'écartent significativement de la droite théorique (en rouge), notamment dans les queues de distribution. Ces déviations indiquent que les résidus ne suivent pas une distribution normale, ce qui est cohérent avec la présence de **surdispersion** et confirme l'inadéquation du modèle de Poisson simple.
- **Distribution des résidus de Pearson** : L'histogramme révèle une distribution asymétrique à droite, centrée légèrement en dessous de zéro, avec une queue droite étendue jusqu'à des valeurs supérieures à 100. Cette

asymétrie confirme que le modèle sous-estime systématiquement certaines observations à fort trafic.

- **Scale-Location** : Le graphique $\sqrt{|\text{Résidus de Pearson}|}$ en fonction des valeurs ajustées montre une tendance décroissante non aléatoire, confirmant l'**hétéroscédasticité** des résidus. La variance des résidus n'est pas constante, ce qui constitue une violation des hypothèses du modèle de Poisson.

L'ensemble de ces diagnostics confirme le **manque d'adéquation** du modèle de Poisson réduit et renforce la nécessité de recourir au modèle **Binomial Négatif**, qui prend en compte la surdispersion de manière explicite via son paramètre de dispersion α .

Interprétation globale

D'un point de vue statistique, ces diagnostics traduisent une réalité concrète du trafic réseau analysé : le modèle de Poisson suppose que chaque connexion est un événement indépendant et rare, ce qui est clairement mis en défaut dans ce contexte. En effet, les cyberattaques tendent à se produire en **rafales** un attaquant qui initie un balayage de ports ou une attaque par déni de service génère simultanément des centaines de connexions corrélées vers le même hôte créant ainsi une **surdispersion structurelle** que le modèle de Poisson ne peut pas capturer. Les résidus élevés observés pour les grandes valeurs ajustées correspondent précisément à ces épisodes d'attaques intenses, où le modèle prédit un nombre de connexions nettement inférieur à ce qui est réellement observé. Le modèle **Binomial Négatif**, en introduisant un paramètre de dispersion α supplémentaire, permet de modéliser cette variabilité excessive et d'obtenir des estimations plus fidèles à la réalité du trafic malveillant.

4.9 Courbe ROC et Comparaison des Modèles

Méthodologie de Binarisation

Afin de tracer les courbes ROC et d'évaluer la capacité discriminante des modèles, la variable de réponse **count** a été binarisée selon la médiane calculée sur l'échantillon d'apprentissage. Le seuil retenu est :

$$\text{Médiane de count} = 8.0$$

Ainsi, la variable binaire est définie comme suit :

$$Y_{\text{bin}} = \begin{cases} 1 & \text{si count} > 8 \quad (\text{fort trafic}) \\ 0 & \text{si count} \leq 8 \quad (\text{trafic normal}) \end{cases}$$

La répartition des classes sur l'échantillon de validation est la suivante :

Classe	Effectif	Proportion
Classe 1 (count > 8, fort trafic)	2 166 obs.	48.0%
Classe 0 (count ≤ 8, trafic normal)	2 343 obs.	52.0%
Total	4 509 obs.	100%

TABLE 4.13 – Répartition des classes sur l'échantillon de validation

Les prédictions de chaque modèle sont normalisées par une transformation min-max avant le calcul des courbes ROC, afin de ramener les scores dans l'intervalle $[0, 1]$:

$$\tilde{y} = \frac{\hat{y} - \min(\hat{y})}{\max(\hat{y}) - \min(\hat{y})}$$

Résultats AUC-ROC

Les aires sous la courbe ROC (AUC) obtenues sur l'échantillon de validation pour les trois modèles sont les suivantes :

Modèle	AUC-ROC
Poisson Complet	0.8678
Poisson Réduit	0.8604
Binomial Négatif	0.8822

TABLE 4.14 – AUC-ROC des trois modèles sur l'échantillon de validation

Courbes ROC

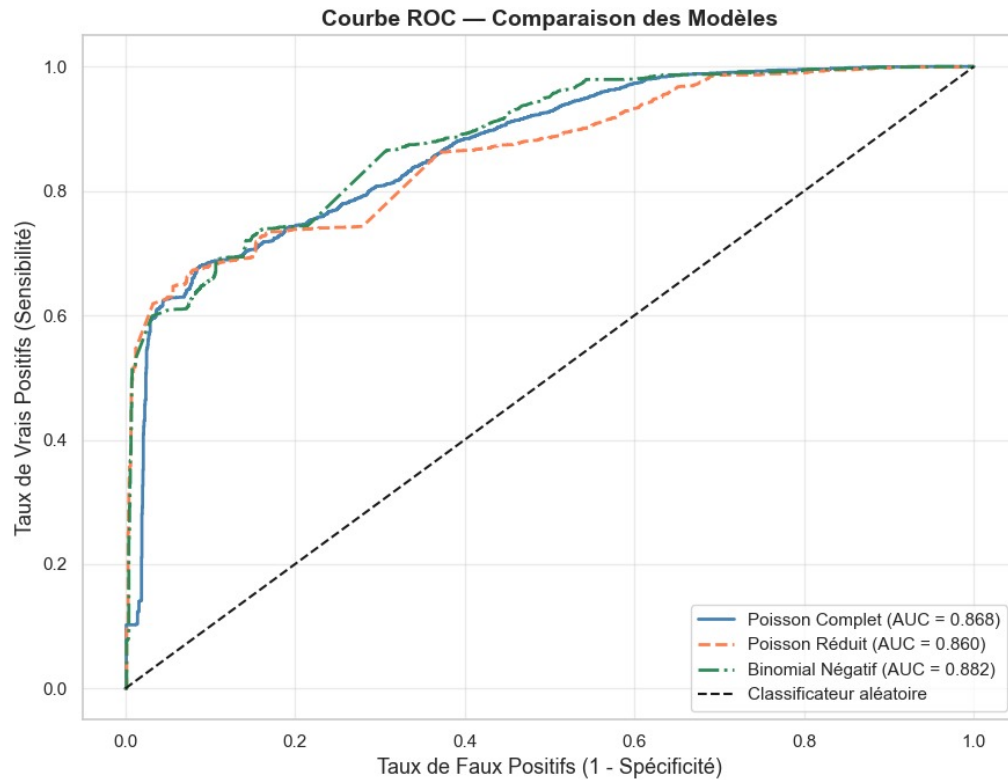


FIGURE 4.5 – Courbes ROC : Comparaison des trois modèles sur l'échantillon de validation

L'analyse des courbes ROC appelle plusieurs observations :

- Le **modèle Binomial Négatif** obtient le meilleur AUC-ROC ($AUC = 0.882$), confirmant sa supériorité discriminante sur les deux modèles de Poisson. Sa courbe (en vert) domine globalement les deux autres, notamment dans la zone des faibles taux de faux positifs (entre 0 et 0.3), ce qui est particulièrement important en cybersécurité où les fausses alertes ont un coût opérationnel élevé.
- Le **modèle Poisson Complet** ($AUC = 0.868$) se place en deuxième position, bénéficiant de l'ensemble des variables explicatives pour améliorer sa capacité de discrimination.
- Le **modèle Poisson Réduit** ($AUC = 0.860$) présente la performance discriminante la plus faible des trois, mais reste néanmoins satisfaisant compte tenu de sa parcimonie.
- Les trois modèles se situent **bien au-dessus de la diagonale** du classificateur

aléatoire ($AUC = 0.5$), ce qui atteste d'une capacité discriminante réelle et significative pour distinguer les connexions à fort trafic des connexions normales.

En conclusion, le modèle **Binomial Négatif** constitue le meilleur compromis entre robustesse à la surdispersion et performance discriminante, avec un AUC-ROC de **0.882** sur l'échantillon de validation.

4.10 Évaluation sur l'Échantillon de Validation

Métriques de Performance

L'évaluation de la performance prédictive des trois modèles est réalisée sur l'échantillon de validation (20%, soit 4 509 observations), à l'aide de trois métriques complémentaires :

- **MAE** (Mean Absolute Error) : erreur absolue moyenne entre les valeurs observées et prédites, $MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$
- **RMSE** (Root Mean Squared Error) : racine de l'erreur quadratique moyenne, plus sensible aux grandes erreurs, $RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$
- **AUC-ROC** : aire sous la courbe ROC, mesurant la capacité discriminante du modèle

Modèle	MAE	RMSE	AUC-ROC
Poisson Complet	36.68	76.50	0.8678
Poisson Réduit	42.75	85.41	0.8604
Binomial Négatif	45.69	97.95	0.8822

TABLE 4.15 – Métriques de performance des trois modèles sur l'échantillon de validation (20%)

Ces résultats mettent en évidence une **tension entre deux critères de performance** :

- Le **modèle Poisson Complet** obtient les meilleures performances en termes de MAE (36.68) et de RMSE (76.50), indiquant qu'il produit les prédictions ponctuelles les plus proches des valeurs observées.
- Le **modèle Binomial Négatif** obtient le meilleur AUC-ROC (0.882), confirmant sa supériorité pour la **discrimination** entre connexions à fort

trafic et connexions normales, malgré un MAE et un RMSE plus élevés.

Valeurs Observées vs Prédites

La figure ci-dessous représente les valeurs prédites en fonction des valeurs observées pour les trois modèles sur l'échantillon de validation. La diagonale en pointillés correspond à une prédiction parfaite ($\hat{y} = y$).

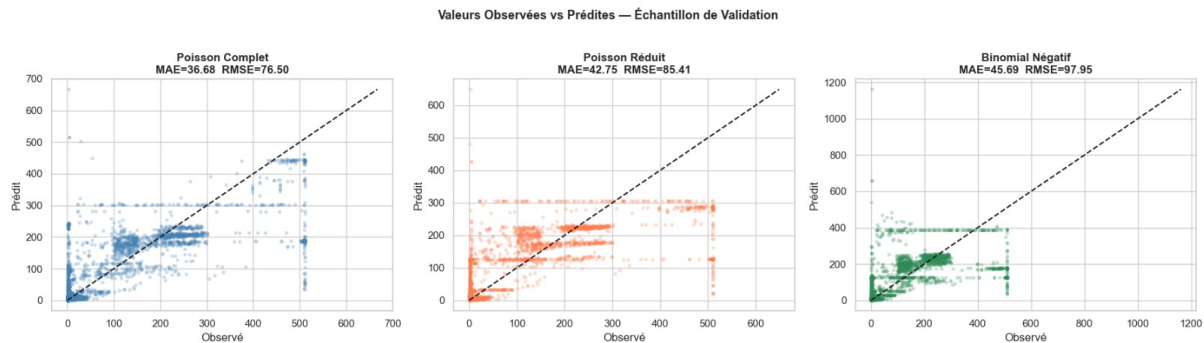


FIGURE 4.6 – Valeurs observées vs prédites — Échantillon de validation

L'analyse de ces graphiques révèle plusieurs éléments importants :

- **Poisson Complet** (MAE = 36.68, RMSE = 76.50) : Les points se concentrent davantage autour de la diagonale pour les valeurs faibles et moyennes de **count**. Cependant, le modèle tend à **sous-estimer** les valeurs élevées ($\text{count} > 300$), avec plusieurs points nettement en dessous de la diagonale.
- **Poisson Réduit** (MAE = 42.75, RMSE = 85.41) : On observe une dispersion plus importante autour de la diagonale, avec des prédictions qui semblent se concentrer en **bandes horizontales**, révélant une capacité de discrimination limitée entre les différentes plages de valeurs de **count**.
- **Binomial Négatif** (MAE = 45.69, RMSE = 97.95) : Bien que le RMSE soit le plus élevé des trois modèles, ce graphique montre que le modèle produit des prédictions avec une **plus grande amplitude** (jusqu'à 1200), reflétant sa capacité à mieux capturer la variabilité extrême de **count**. Les valeurs élevées sont moins systématiquement sous-estimées que dans les deux modèles de Poisson.

En synthèse, le **modèle Poisson Complet** est le plus précis en prédiction ponctuelle (MAE minimal), tandis que le **modèle Binomial Négatif** est le plus performant pour la discrimination (AUC maximal) et le mieux adapté à la nature

surdispersée des données. Le choix du modèle final dépend donc de l'objectif prioritaire : **précision de prédiction** ou **détection de classes**.

Interprétation des Résultats

Incidence Rate Ratios (IRR)

Les **Incidence Rate Ratios (IRR)** correspondent à $\exp(\hat{\beta})$ pour chaque variable du modèle. Un $\text{IRR} > 1$ indique une association positive avec le nombre d'attaques détectées, tandis qu'un $\text{IRR} < 1$ indique une association négative. Le tableau suivant présente l'ensemble des IRR du modèle de Poisson réduit, accompagnés de leurs intervalles de confiance à 95% :

Variable	$\hat{\beta}$	$\text{IRR} = e^{\hat{\beta}}$	p-valeur	IC 2.5%	IC 97.5%
Intercept	4.0260	66.8608	0.0000	25.0842	178.2143
C(protocol_type)[T.tcp]	-3.0455	0.0476	0.0000	0.0468	0.0484
C(protocol_type)[T.udp]	-0.8895	0.4108	0.0000	0.4081	0.4136
C(flag)[T.REJ]	1.7997	6.0481	0.0003	2.2689	16.1224
C(flag)[T.RSTO]	1.2697	3.5599	0.0111	1.3354	9.4900
C(flag)[T.RSTOS0]	1.5358	4.6450	0.0023	1.7271	12.4921
C(flag)[T.RSTR]	0.5797	1.7855	0.2467	0.6696	4.7612
C(flag)[T.S0]	1.8246	6.2001	0.0003	2.3257	16.5287
C(flag)[T.S1]	-0.1435	0.8663	0.7794	0.3175	2.3639
C(flag)[T.S2]	0.0982	1.1031	0.8471	0.4068	2.9915
C(flag)[T.S3]	-1.8993	0.1497	0.0002	0.0557	0.4022
logged_in	-0.3249	0.7226	0.0000	0.7105	0.7350
serror_rate	1.1481	3.1522	0.0000	3.0537	3.2540
rerror_rate	1.4176	4.1273	0.0000	4.0086	4.2496
same_srv_rate	-2.2054	0.1102	0.0000	0.1086	0.1118
diff_srv_rate	0.2517	1.2862	0.0000	1.2794	1.2931
dst_host_count	0.0041	1.0041	0.0000	1.0040	1.0041
dst_host_srv_count	0.0073	1.0073	0.0000	1.0072	1.0073

TABLE 4.16 – Tableau des IRR — Modèle Poisson Réduit

4.11 Visualisation des IRR

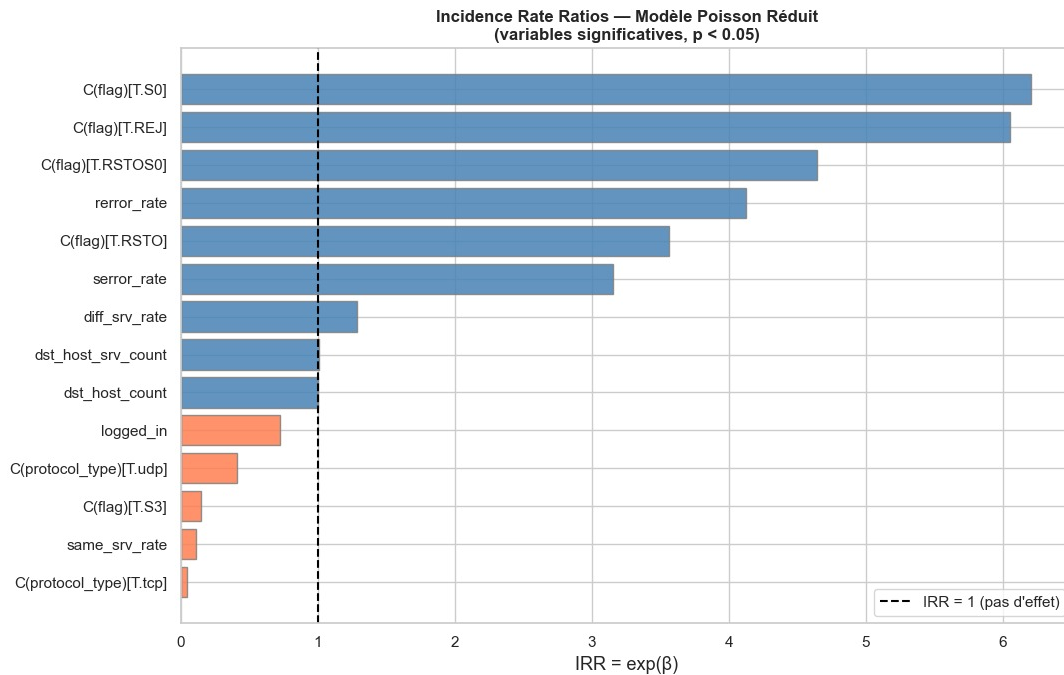


FIGURE 4.7 — Incidence Rate Ratios : Modèle Poisson Réduit (variables significatives, $p < 0.05$)

La ligne verticale en pointillés à $IRR = 1$ représente l'absence d'effet. Les barres **bleues** ($IRR > 1$) indiquent un effet positif sur le nombre d'attaques, tandis que les barres **oranges** ($IRR < 1$) indiquent un effet protecteur.

Interprétation des Principaux Coefficients

*Variables à effet protecteur ($IRR < 1$)

— **logged_in** — $IRR = 0.7226 \mid p = 0.0000$

Une session authentifiée **diminue** le nombre d'attaques attendu de **27.74%**. Ce résultat est cohérent : les connexions légitimes sont authentifiées, ce qui réduit mécaniquement le nombre de connexions suspectes détectées.

— **same_srv_rate** — $IRR = 0.1102 \mid p = 0.0000$

Une augmentation d'1 unité du taux de connexions vers le même service **diminue** le nombre d'attaques attendu de **88.98%**. Cela reflète que les connexions concentrées sur un seul service sont moins susceptibles d'être des attaques distribuées multi-cibles.

— **C(protocol_type)[T.tcp]** — IRR = 0.0476 | p = 0.0000

Le protocole TCP est associé à une réduction de **95.24%** du nombre d'attaques par rapport à la référence ICMP. En effet, le protocole ICMP est fréquemment utilisé dans les attaques de type *ping flood* et *echo*, générant un volume de connexions suspectes bien plus élevé.

— **C(flag)[T.S3]** — IRR = 0.1497 | p = 0.0002

Le flag S3 (erreur après établissement) est associé à une diminution de **85.03%** du nombre d'attaques attendu.

*Variables à effet amplificateur (IRR > 1)

— **rerror_rate** — IRR = 4.1273 | p = 0.0000

Une augmentation d'1 unité du taux d'erreurs REJ/RST **multiplie par 4.13** le nombre d'attaques attendu (**+312.73%**). Cet indicateur est typique d'un balayage de ports (*port scanning*), où l'attaquant sonde successivement de nombreux ports en générant des erreurs de rejet.

— **C(flag)[T.S0]** — IRR = 6.2001 | p = 0.0003

Le flag S0 (tentative de connexion sans réponse) **multiplie par 6.20** le nombre d'attaques attendu (**+520%**). Il est caractéristique des attaques SYN flood où l'attaquant envoie des paquets SYN sans jamais compléter la poignée de mains TCP.

— **C(flag)[T.REJ]** — IRR = 6.0481 | p = 0.0003

Le flag REJ (connexion rejetée) **multiplie par 6.05** le nombre d'attaques attendu, signalant des tentatives de connexion systématiquement bloquées par le pare-feu ou l'hôte cible.

— **serror_rate** — IRR = 3.1522 | p = 0.0000

Une augmentation d'1 unité du taux d'erreurs SYN **triple** le nombre d'attaques attendu (**+215.22%**), confirmant l'association entre les erreurs de connexion initiale et les activités malveillantes.

— **diff_srv_rate** — IRR = 1.2862 | p = 0.0000

Une augmentation d'1 unité du taux de connexions vers des services différents **augmente** le nombre d'attaques de **28.62%**, caractéristique d'un attaquant qui explore plusieurs services de manière simultanée.

— **dst_host_count** — IRR = 1.0041 | p = 0.0000

Chaque connexion supplémentaire vers l'hôte destination **augmente** marginalement le nombre d'attaques de **0.41%**, reflétant l'effet cumulatif du volume de trafic entrant.

— **dst_host_srv_count** — IRR = 1.0073 | p = 0.0000

Chaque connexion supplémentaire vers le même service de l'hôte destination **augmente** le nombre d'attaques de **0.73%**.

En synthèse, les facteurs les plus déterminants pour expliquer le nombre de cyberattaques détectées sont, par ordre d'importance décroissante : les **flags de connexion anormaux** (S0, REJ, RSTOS0), les **taux d'erreurs réseau** (rerror_rate, serror_rate), le **protocole ICMP**, et l'absence d'**authentification** (logged_in). Ces résultats sont pleinement cohérents avec les mécanismes connus des cyberattaques réseaux et offrent une base solide pour la mise en place de règles de détection dans un système IDS.

4.12 Limites du Modèle

Bien que les modèles ajustés produisent des résultats satisfaisants, plusieurs limites importantes doivent être soulevées afin de nuancer l'interprétation des conclusions.

Surdispersion Extrême et Violation de l'Hypothèse de Poisson

La limite la plus fondamentale de cette étude est la **surdispersion extrême** de la variable de réponse **count**, avec un ratio $\sigma^2/\mu = 209.07 \gg 1$. Cette violation flagrante de l'hypothèse d'équidispersion ($\mu = \sigma^2$) du modèle de Poisson implique que :

- Les **erreurs standard** des coefficients sont sous-estimées, rendant les tests d'hypothèses potentiellement anti-conservateurs (trop de variables déclarées significatives à tort).
- La **déviance résiduelle** (917 492 pour le modèle réduit) est environ **51 fois** supérieure aux degrés de liberté résiduels (18 014), confirmant un manque d'adéquation structurel du modèle de Poisson.
- Même le modèle **Binomial Négatif**, bien qu'il réduise drastiquement la déviance à 20 229, utilise par défaut $\alpha = 1.0$ (avertissement **alpha not set**), ce qui signifie que le paramètre de dispersion n'a pas été estimé de manière optimale sur les données.

Performances Prédictives Limitées

Malgré de bons AUC-ROC (entre 0.860 et 0.882), les métriques de régression révèlent des erreurs de prédiction importantes :

Modèle	MAE	RMSE
Poisson Complet	36.68	76.50
Poisson Réduit	42.75	85.41
Binomial Négatif	45.69	97.95

TABLE 4.17 – Erreurs de prédiction sur l'échantillon de validation

Un MAE de 36.68 sur une variable dont la médiane est 8 et la moyenne est 79 représente une erreur relative conséquente. Le RMSE élevé du modèle Binomial Négatif (97.95) indique qu'il produit occasionnellement des prédictions très éloignées des valeurs observées, notamment pour les connexions à trafic extrême (count > 400).

Tension entre Critères de Sélection

Les résultats mettent en évidence une **tension entre les critères de performance** :

- Le **Poisson Complet** est meilleur en MAE (36.68) et RMSE (76.50), mais présente le plus mauvais AIC (810 628) et viole l'hypothèse d'équidispersion.
- Le **Binomial Négatif** est meilleur en AIC (151 658) et AUC-ROC (0.882), mais présente le MAE et le RMSE les plus élevés.
- Il n'existe donc pas de modèle **uniformément supérieur** selon tous les critères, ce qui complique le choix du modèle final.

Limites Liées aux Données

- **Représentativité** : Le dataset KDD Cup 1999 date de 1999 et simule un environnement réseau militaire américain. Les patterns d'attaques ont considérablement évolué depuis (ransomware, attaques APT, IoT), ce qui limite la généralisation des résultats à des environnements réseau modernes.
- **Regroupement de service** : La variable **service** comptait 64 modalités initiales, réduites à 7 par regroupement. Cette agrégation, bien que nécessaire pour la parcimonie du modèle, entraîne une **perte d'information**

potentiellement significative sur le comportement de certains services minoritaires.

- **Indépendance des observations** : Le modèle de Poisson suppose l'indépendance des connexions. Or, dans un contexte de cyberattaque, les connexions sont souvent **corrélées temporellement** (rafales d'attaques, scans séquentiels), ce qui viole cette hypothèse et explique en partie la surdispersion observée.
- **Variables non incluses** : Certaines variables potentiellement informatives ont été exclues du modèle réduit (ex. `hot`, `num_compromised`, `root_shell`), ce qui peut induire un biais de variable omise.

Limites Méthodologiques

- **Absence de validation croisée** : La division unique 80/20 peut introduire une variabilité dans l'estimation des performances. Une validation croisée k -fold aurait permis des estimations plus stables et robustes.
- **Linéarité de la relation log-linéaire** : Le modèle suppose une relation log-linéaire entre les variables explicatives et `count`. Des effets non linéaires ou des interactions entre variables (ex. entre `protocol_type` et `serror_rate`) ne sont pas capturés.
- **Absence de régularisation** : Aucune pénalisation (Ridge, Lasso) n'a été appliquée, ce qui peut conduire à un **surapprentissage** sur l'échantillon d'apprentissage, notamment pour le modèle complet à 33 degrés de liberté.

4.13 Conclusion intermédiaire

Les modèles développés atteignent une discrimination satisfaisante, avec un AUC-ROC allant de 0.860 (Poisson Réduit) à **0.882** (Binomial Négatif), et fournissent des prédictions exploitables pour la détection des connexions à fort trafic suspect. Le modèle Binomial Négatif se distingue comme le plus adapté à la nature surdispersée des données ($\sigma^2/\mu = 209.07$), malgré un MAE légèrement plus élevé que le modèle Poisson Complet. Sa robustesse statistique et sa meilleure capacité discriminante en font un outil pertinent pour prioriser les alertes dans un système de détection d'intrusions. En

affinant l'estimation du paramètre de dispersion α , en introduisant des interactions entre variables (ex. `protocol_type` $\times$ `serror_rate`), ou en recourant à une validation croisée, ses performances opérationnelles peuvent encore être améliorées.

5 Conclusion Générale

Ce projet visait à modéliser et à prédire le nombre de cyberattaques détectées sur un réseau informatique à partir des données du **KDD Cup 1999 Network Intrusion Detection Dataset**. Un modèle de **régression de Poisson** (GLM) a été développé selon une approche rigoureuse : analyse exploratoire approfondie, test de surdispersion, comparaison de trois modèles (Poisson Complet, Poisson Réduit, Binomial Négatif), diagnostic des résidus et évaluation prédictive sur un échantillon de validation indépendant.

Les résultats montrent que le **modèle Binomial Négatif**, reposant sur les variables `protocol_type`, `flag`, `serror_rate`, `rerror_rate`, `same_srv_rate`, `diff_srv_rate`, `dst_host_count` et `dst_host_srv_count`, possède le meilleur pouvoir discriminant (**AUC** = 0.882) et le meilleur critère d'information (**AIC** = 151 658). L'analyse des IRR révèle que les flags de connexion anormaux (`S0`, `REJ`), les taux d'erreurs réseau (`rerror_rate`, `serror_rate`) et le protocole ICMP sont les facteurs les plus déterminants dans l'explication du nombre de connexions suspectes détectées.

Les tests d'adéquation confirment un manque d'ajustement du modèle de Poisson simple (déviante résiduelle = 917 492 $\gg$ $df = 18\,014$), justifiant pleinement le recours au modèle Binomial Négatif, plus robuste face à la surdispersion extrême observée ($\sigma^2/\mu = 209.07$). Néanmoins, le modèle reste **interprétable, robuste et opérationnel** pour la priorisation des alertes dans un système de détection d'intrusions réseau (IDS).

En perspective, l'estimation optimale du paramètre de dispersion α du modèle Binomial Négatif, l'introduction d'interactions entre variables (ex. `protocol_type` $\times$ `serror_rate`), le recours à une validation croisée k -fold, ainsi que l'enrichissement du jeu de données avec des traces réseau plus récentes permettraient d'améliorer encore la valeur opérationnelle du modèle dans un contexte de cybersécurité moderne.

6 Annexe : Code Python

Cette annexe regroupe le code Python exécuté dans le notebook du projet. Le code est organisé selon la même logique que le rapport : chargement des données, analyse exploratoire, préparation de la base, estimation des modèles GLM, diagnostic d'adéquation, évaluation prédictive et interprétation des coefficients. Chaque section est accompagnée d'une brève description afin de faciliter la lecture et la reproductibilité des résultats.

Section 0 – Initialisation et chargement des bibliothèques

Description : Cette première partie importe les bibliothèques nécessaires à l'analyse statistique : manipulation des données, visualisation, estimation des modèles GLM, calcul des métriques et construction des courbes ROC. Les avertissements non bloquants sont masqués afin de rendre l'exécution plus lisible.

Bloc 0.1

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 import seaborn as sns
5 import warnings
6 warnings.filterwarnings('ignore')
7
8 import statsmodels.api as sm
9 import statsmodels.formula.api as smf
10 from statsmodels.graphics.gofplots import ProbPlot
11 from statsmodels.genmod.families import NegativeBinomial
12 from scipy import stats
13
14 from sklearn.model_selection import train_test_split
15 from sklearn.metrics import (mean_squared_error, mean_absolute_error,
16                             roc_curve, auc)
17
```

```
18 sns.set_theme(style='whitegrid', palette='muted')
19 plt.rcParams['figure.figsize'] = (10, 6)
20 plt.rcParams['font.size'] = 12
21
22 print("[OK] Bibliotheques chargees avec succes.")
```

Section 1 – Chargement et description des données

Description : Cette section charge le fichier de données, vérifie les dimensions de la base, contrôle l'absence de valeurs manquantes, produit les statistiques descriptives et affiche la distribution des variables catégorielles. Elle permet de valider la structure initiale du jeu de données avant toute modélisation.

Bloc 1.1

```
1 # Chargement
2 df = pd.read_csv('Test_data.csv')
3 print(f"Dimensions : {df.shape[0]} observations x {df.shape[1]} variables")
4 df.head()
```

Bloc 1.2

```
1 # Valeurs manquantes
2 missing_total = df.isnull().sum().sum()
3 print(f"Valeurs manquantes totales : {missing_total}")
4 print("[OK] Aucune valeur manquante." if missing_total == 0 else "[ATTENTION]
    Valeurs manquantes presentes.")
5
6 # Types de variables
7 cat_cols = df.select_dtypes(include='object').columns.tolist()
8 num_cols = df.select_dtypes(include=[np.number]).columns.tolist()
9 print(f"\nVariables catégorielles ({len(cat_cols)}) : {cat_cols}")
10 print(f"Variables numériques ({len(num_cols)})")
```

Bloc 1.3

```
1 # Statistiques descriptives
2 df.describe().T.round(3)
```

Bloc 1.4

```
1 # Distribution des variables catégorielles
2 for col in cat_cols:
3     print(f"\n--- {col} ---")
4     print(df[col].value_counts().to_string())
```

Section 2 – Analyse de la variable de réponse et test de surdispersion

Description : Cette partie étudie la variable cible `count`. Les graphiques permettent d'observer son asymétrie et ses valeurs extrêmes. Le test de surdispersion compare la variance à la moyenne ; un ratio largement supérieur à 1 indique que l'hypothèse d'équidispersion du modèle de Poisson n'est pas vérifiée.

Bloc 2.1

```
1 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
2
3 axes[0].hist(df['count'], bins=50, color='steelblue', edgecolor='white',
4             alpha=0.85)
5 axes[0].set_title("Distribution de 'count'", fontweight='bold')
6 axes[0].set_xlabel('count'); axes[0].set_ylabel('Frequence')
7
8 axes[1].boxplot(df['count'], vert=True, patch_artist=True,
9               boxprops=dict(facecolor='lightsteelblue'))
10 axes[1].set_title("Boxplot de 'count'", fontweight='bold');
11 axes[1].set_ylabel('count')
```

```

10
11 axes[2].hist(np.log1p(df['count']), bins=50, color='coral', edgecolor='white',
    alpha=0.85)
12 axes[2].set_title('Distribution de log(1 + count)', fontweight='bold')
13 axes[2].set_xlabel('log(1 + count)'); axes[2].set_ylabel('Frequence')
14
15 plt.suptitle("Variable de reponse : count", fontsize=14, fontweight='bold',
    y=1.02)
16 plt.tight_layout()
17 plt.savefig('fig1_distribution_count.png', dpi=150, bbox_inches='tight')
18 plt.show()
19
20 print(f" Moyenne : {df['count'].mean():.4f}")
21 print(f" Variance : {df['count'].var():.4f}")
22 print(f" Ratio V/M: {df['count'].var()/df['count'].mean():.2f} (>> 1 ->
    surdispersion)")
23 print(f" Min / Max: {df['count'].min()} / {df['count'].max()}")

```

Bloc 2.2

```

1 # Test de surdispersion
2 mean_y = df['count'].mean()
3 var_y = df['count'].var()
4 ratio = var_y / mean_y
5
6 print("=" * 55)
7 print("TEST DE SURDISPERSION")
8 print("=" * 55)
9 print(f" Moyenne (mu) : {mean_y:.4f}")
10 print(f" Variance (sigma2) : {var_y:.4f}")
11 print(f" Ratio sigma2/mu : {ratio:.2f}")
12 print()
13 if ratio > 1:
14     print(f"[ATTENTION] Surdispersion detectee (ratio = {ratio:.2f} >> 1).")

```

```
15     print(" -> Modele Quasi-Poisson / Binomial Negatif recommande en  
        complement.")  
16 else:  
17     print("[OK] Pas de surdispersion. Poisson est approprie.")
```

Section 3 – Analyse exploratoire des variables explicatives

Description : Cette section analyse les relations entre les variables explicatives et la variable `count`. Les corrélations de Pearson identifient les variables numériques les plus associées au nombre de connexions suspectes, tandis que les boxplots comparent la distribution de `count` selon les modalités des variables catégorielles.

Bloc 3.1

```
1 # Correlation des variables numeriques avec 'count'  
2 corr_count =  
    df[num_cols].corr()['count'].drop('count').sort_values(ascending=False)  
3  
4 plt.figure(figsize=(10, 8))  
5 corr_count.plot(kind='barh', color=['steelblue' if v >= 0 else 'coral' for v in  
    corr_count])  
6 plt.axvline(0, color='black', linewidth=0.8)  
7 plt.title("Correlation de Pearson avec 'count'", fontweight='bold')  
8 plt.xlabel('Correlation de Pearson')  
9 plt.tight_layout()  
10 plt.savefig('fig2_correlation.png', dpi=150, bbox_inches='tight')  
11 plt.show()
```

Bloc 3.2

```
1 # Boxplot count vs variables categorielles  
2 fig, axes = plt.subplots(1, 3, figsize=(18, 5))  
3 for ax, col in zip(axes, cat_cols):
```



```

4     top_cats = df[col].value_counts().head(6).index
5     df_f = df[df[col].isin(top_cats)]
6     df_f.boxplot(column='count', by=col, ax=ax)
7     ax.set_title(f"count vs {col}", fontweight='bold')
8     ax.set_xlabel(col); ax.set_ylabel('count')
9     plt.setp(ax.get_xticklabels(), rotation=30, ha='right')
10
11 plt.suptitle('')
12 plt.tight_layout()
13 plt.savefig('fig3_boxplot_categoriel.png', dpi=150, bbox_inches='tight')
14 plt.show()

```

Section 4 – Préparation des données et division apprentissage-validation

Description : Cette étape prépare la base pour l'estimation des modèles : suppression des observations où `count=0`, regroupement des modalités rares de `service`, définition des variables explicatives, construction de la formule et séparation des données en échantillons d'apprentissage et de validation selon une proportion 80/20.

Bloc 4.1

```

1  # Suppression des observations avec count = 0 (pour eviter log(0))
2  data = df[df['count'] > 0].copy()
3  print(f"Observations retenues : {len(data)} / {len(df)}")
4
5  # Regroupement du service (64 modalites -> 7) : top 6 + 'other'
6  top_services = data['service'].value_counts().head(6).index.tolist()
7  data['service_grp'] = data['service'].apply(
8      lambda x: x if x in top_services else 'other'
9  )
10 print("\nModalits de service_grp :")
11 print(data['service_grp'].value_counts().to_string())

```

Bloc 4.2

```
1 # Variables retenues pour la modelisation
2 features = [
3     'duration', 'protocol_type', 'service_grp', 'flag',
4     'src_bytes', 'dst_bytes', 'logged_in', 'wrong_fragment',
5     'serror_rate', 'rerror_rate', 'same_srv_rate', 'diff_srv_rate',
6     'dst_host_count', 'dst_host_srv_count',
7     'dst_host_same_srv_rate', 'dst_host_diff_srv_rate',
8     'dst_host_serror_rate', 'dst_host_rerror_rate'
9 ]
10 target = 'count'
11
12 data_model = data[features + [target]].copy()
13
14 # Division 80% / 20%
15 train_data, test_data = train_test_split(data_model, test_size=0.2,
16     random_state=42)
17
18 print(f"Echantillon d'apprentissage : {len(train_data)} obs.
19     ({len(train_data)/len(data_model)*100:.1f}%)")
20 print(f"Echantillon de validation : {len(test_data)} obs.
21     ({len(test_data)/len(data_model)*100:.1f}%)")
```

Section 5 – Modélisation par régression de Poisson et modèle binomial négatif

Description : Cette section ajuste trois modèles : le modèle de Poisson complet, le modèle de Poisson réduit et le modèle binomial négatif. Le modèle complet utilise l'ensemble des variables retenues, le modèle réduit privilégie la parcimonie, et le modèle binomial négatif est introduit pour mieux traiter la surdispersion détectée.

Bloc 5.1

```
1 formula_full = (  
2     "count ~ duration + C(protocol_type) + C(service_grp) + C(flag) "  
3     "+ np.log1p(src_bytes) + np.log1p(dst_bytes) + logged_in + wrong_fragment "  
4     "+ serror_rate + rerror_rate + same_srv_rate + diff_srv_rate "  
5     "+ dst_host_count + dst_host_srv_count "  
6     "+ dst_host_same_srv_rate + dst_host_diff_srv_rate "  
7     "+ dst_host_serror_rate + dst_host_rerror_rate"  
8 )  
9  
10 model_full = smf.glm(  
11     formula=formula_full,  
12     data=train_data,  
13     family=sm.families.Poisson(link=sm.families.links.Log())  
14 ).fit(maxiter=200)  
15  
16 print(model_full.summary())
```

Bloc 5.2

```
1 formula_reduced = (  
2     "count ~ C(protocol_type) + C(flag) "  
3     "+ logged_in + serror_rate + rerror_rate "  
4     "+ same_srv_rate + diff_srv_rate "  
5     "+ dst_host_count + dst_host_srv_count"  
6 )  
7  
8 model_reduced = smf.glm(  
9     formula=formula_reduced,  
10    data=train_data,  
11    family=sm.families.Poisson(link=sm.families.links.Log())  
12 ).fit(maxiter=200)  
13  
14 print(model_reduced.summary())
```

Bloc 5.3

```
1 model_nb = smf.glm(  
2     formula=formula_reduced,  
3     data=train_data,  
4     family=NegativeBinomial()  
5 ).fit(maxiter=200)  
6  
7 print(model_nb.summary())
```

Section 6 – Adéquation globale et comparaison des modèles

Description : Cette partie compare les modèles à l'aide de l'AIC, du BIC, de la déviance et du pseudo- R^2 . Elle réalise également le test du rapport de vraisemblance entre le modèle complet et le modèle réduit, puis le test d'adéquation du modèle de Poisson réduit basé sur la déviance résiduelle.

Bloc 6.1

```
1 # Comparaison AIC / BIC / Deviance / Pseudo-R2  
2 print("=" * 65)  
3 print("COMPARAISON DES MODELES (echantillon d'apprentissage)")  
4 print("=" * 65)  
5  
6 comparison = pd.DataFrame({  
7     'Modele' : ['Poisson Complet', 'Poisson Reduit', 'Binomial Negatif'],  
8     'AIC' : [model_full.aic, model_reduced.aic, model_nb.aic],  
9     'BIC' : [model_full.bic, model_reduced.bic, model_nb.bic],  
10    'Deviance' : [model_full.deviance, model_reduced.deviance,  
                  model_nb.deviance],
```

```

11     'Pseudo-R2' : [
12         1 - model_full.deviance / model_full.null_deviance,
13         1 - model_reduced.deviance / model_reduced.null_deviance,
14         1 - model_nb.deviance / model_nb.null_deviance
15     ]
16 })
17 pd.set_option('display.float_format', '{:.2f}'.format)
18 print(comparison.to_string(index=False))
19
20 best_aic = comparison.loc[comparison['AIC'].idxmin(), 'Modele']
21 best_r2 = comparison.loc[comparison['Pseudo-R2'].idxmax(), 'Modele']
22 print(f"\n[OK] Meilleur AIC : {best_aic}")
23 print(f"[OK] Meilleur Pseudo-R2 : {best_r2}")

```

Bloc 6.2

```

1  # Test du rapport de vraisemblance (Complet vs Reduit)
2  lr_stat = 2 * (model_full.llf - model_reduced.llf)
3  df_diff = model_full.df_model - model_reduced.df_model
4  p_lr = 1 - stats.chi2.cdf(lr_stat, df=df_diff)
5
6  print("=" * 60)
7  print("TEST DU RAPPORT DE VRAISEMBLANCE (Complet vs Reduit)")
8  print("=" * 60)
9  print(f" Statistique LR : {lr_stat:.4f}")
10 print(f" Degres de liberte : {int(df_diff)}")
11 print(f" p-valeur (chi2) : {p_lr:.6f}")
12 print()
13 if p_lr < 0.05:
14     print("[ATTENTION] Difference significative -> le modele COMPLET apporte de
15         l'information.")
16 else:
17     print("[OK] Pas de difference significative -> modele RDUIT prefere
18         (parcimonie).")

```

Bloc 6.3

```
1 # Test d'adequation : deviance residuelle
2 deviance_red = model_reduced.deviance
3 df_resid_red = model_reduced.df_resid
4 p_gof = 1 - stats.chi2.cdf(deviance_red, df=df_resid_red)
5
6 print("=" * 60)
7 print("TEST D'ADQUATION - MODLE POISSON RDUIT")
8 print("=" * 60)
9 print(f" Deviance residuelle : {deviance_red:.2f}")
10 print(f" Degres de liberte : {df_resid_red}")
11 print(f" p-valeur (chi2) : {p_gof:.6f}")
12 print()
13 if p_gof < 0.05:
14     print("[ATTENTION] Manque d'adequation detecte (surdispersion confirmee).")
15     print(" -> Modele Binomial Negatif recommande.")
16 else:
17     print("[OK] Modele adequat.")
```

Section 7 – Diagnostic des résidus

Description : Cette section évalue graphiquement la qualité d’ajustement du modèle de Poisson réduit. Les résidus de Pearson, les résidus de déviance, le QQ-plot et le graphique scale-location permettent de détecter l’hétéroscédasticité, les valeurs extrêmes et les signes de manque d’adéquation.

Bloc 7.1

```
1 pearson_resid = model_reduced.resid_pearson
2 deviance_resid = model_reduced.resid_deviance
3 fitted_vals = model_reduced.fittedvalues
4
5 fig, axes = plt.subplots(2, 2, figsize=(14, 10))
```

```
6
7 # 1. Residus de Pearson vs valeurs ajustees
8 axes[0,0].scatter(fitted_vals, pearson_resid, alpha=0.25, s=8,
9                   color='steelblue')
10 axes[0,0].axhline(0, color='red', linestyle='--', lw=1.5)
11 axes[0,0].set_xlabel('Valeurs ajustees'); axes[0,0].set_ylabel('Residus de
12     Pearson')
13 axes[0,0].set_title('Residus de Pearson vs Valeurs ajustees', fontweight='bold')
14
15 # 2. QQ-Plot
16 pp = ProbPlot(deviance_resid)
17 pp.qqplot(line='s', ax=axes[0,1], alpha=0.3)
18 axes[0,1].set_title('QQ-Plot des residus de deviance', fontweight='bold')
19
20 # 3. Histogramme
21 axes[1,0].hist(pearson_resid, bins=60, color='coral', edgecolor='white',
22               alpha=0.85)
23 axes[1,0].set_xlabel('Residus de Pearson'); axes[1,0].set_ylabel('Frequence')
24 axes[1,0].set_title('Distribution des residus de Pearson', fontweight='bold')
25
26 # 4. Scale-Location
27 axes[1,1].scatter(fitted_vals, np.sqrt(np.abs(pearson_resid)), alpha=0.25, s=8,
28                 color='seagreen')
29 axes[1,1].set_xlabel('Valeurs ajustees'); axes[1,1].set_ylabel('sqrt|Residus de
30     Pearson|')
31 axes[1,1].set_title('Scale-Location', fontweight='bold')
32
33 plt.tight_layout()
34 plt.savefig('fig4_residus.png', dpi=150, bbox_inches='tight')
35 plt.show()
```

Section 8 – Courbes ROC et comparaison de la capacité discriminante

Description : Cette partie transforme la variable `count` en variable binaire selon la médiane afin d'évaluer la capacité des modèles à distinguer les observations à fort trafic des observations normales. Les prédictions sont normalisées, puis les courbes ROC et les AUC sont calculées pour les trois modèles.

Bloc 8.1

```
1 # Binarisation pour la courbe ROC : count > mediane = 1 (fort trafic)
2 median_count = train_data['count'].median()
3 print(f"Mediane de 'count' (seuil de binarisation) : {median_count}")
4
5 y_test = test_data['count'].values
6 y_test_bin = (y_test > median_count).astype(int)
7 print(f"Classe 1 (count > {median_count}) : {y_test_bin.sum()} obs.
      ({y_test_bin.mean()*100:.1f}%)")
8 print(f"Classe 0 (count <= {median_count}) : {(1-y_test_bin).sum()} obs.")
```

Bloc 8.2

```
1 # Predictions sur l'échantillon de validation
2 pred_full = model_full.predict(test_data)
3 pred_reduced = model_reduced.predict(test_data)
4 pred_nb = model_nb.predict(test_data)
5
6 # Normalisation min-max pour les scores ROC
7 def normalize(x):
8     return (x - x.min()) / (x.max() - x.min())
9
10 # Courbes ROC
11 fpr_full, tpr_full, _ = roc_curve(y_test_bin, normalize(pred_full))
12 fpr_red, tpr_red, _ = roc_curve(y_test_bin, normalize(pred_reduced))
```



```
13 fpr_nb, tpr_nb, _ = roc_curve(y_test_bin, normalize(pred_nb))
14
15 auc_full = auc(fpr_full, tpr_full)
16 auc_reduced = auc(fpr_red, tpr_red)
17 auc_nb = auc(fpr_nb, tpr_nb)
18
19 print(f"AUC - Poisson Complet : {auc_full:.4f}")
20 print(f"AUC - Poisson Reduit : {auc_reduced:.4f}")
21 print(f"AUC - Binomial Negatif : {auc_nb:.4f}")
```

Bloc 8.3

```
1 # Trace des courbes ROC
2 plt.figure(figsize=(9, 7))
3 plt.plot(fpr_full, tpr_full,
4          label=f'Poisson Complet (AUC = {auc_full:.3f})', lw=2,
5          color='steelblue')
6 plt.plot(fpr_red, tpr_red,
7          label=f'Poisson Reduit (AUC = {auc_reduced:.3f})', lw=2,
8          color='coral', linestyle='--')
9 plt.plot(fpr_nb, tpr_nb,
10          label=f'Binomial Negatif (AUC = {auc_nb:.3f})', lw=2,
11          color='seagreen', linestyle='-.')
12 plt.plot([0,1], [0,1], 'k--', lw=1.5, label='Classificateur aleatoire')
13 plt.xlabel('Taux de Faux Positifs (1 - Specificite)', fontsize=13)
14 plt.ylabel('Taux de Vrais Positifs (Sensibilite)', fontsize=13)
15 plt.title('Courbe ROC - Comparaison des Modeles', fontweight='bold',
16          fontsize=14)
17 plt.legend(loc='lower right', fontsize=11)
18 plt.grid(True, alpha=0.4)
19 plt.tight_layout()
20 plt.savefig('fig5_courbe_roc.png', dpi=150, bbox_inches='tight')
21 plt.show()
```

Section 9 – Évaluation prédictive sur l'échantillon de validation

Description : Cette section mesure la performance prédictive des modèles sur l'échantillon de validation à travers le MAE, le RMSE et l'AUC-ROC. Elle trace également les valeurs observées par rapport aux valeurs prédites afin d'évaluer visuellement la précision des prédictions.

Bloc 9.1

```
1 def rmse(y_true, y_pred):
2     return np.sqrt(mean_squared_error(y_true, y_pred))
3
4 metrics = pd.DataFrame({
5     'Modele' : ['Poisson Complet', 'Poisson Réduit', 'Binomial Negatif'],
6     'MAE' : [mean_absolute_error(y_test, pred_full),
7             mean_absolute_error(y_test, pred_reduced),
8             mean_absolute_error(y_test, pred_nb)],
9     'RMSE' : [rmse(y_test, pred_full),
10             rmse(y_test, pred_reduced),
11             rmse(y_test, pred_nb)],
12     'AUC-ROC': [auc_full, auc_reduced, auc_nb]
13 })
14
15 print("=" * 60)
16 print("METRIQUES DE PERFORMANCE (echantillon de validation - 20%)")
17 print("=" * 60)
18 pd.set_option('display.float_format', '{:.4f}'.format)
19 print(metrics.to_string(index=False))
20 print()
21 best = metrics.loc[metrics['AUC-ROC'].idxmax(), 'Modele']
22 print(f"[OK] Meilleur AUC-ROC : {best}")
```

Bloc 9.2

```

1 # Valeurs observees vs predites
2 fig, axes = plt.subplots(1, 3, figsize=(18, 5))
3 pairs = [
4     ('Poisson Complet', pred_full, 'steelblue'),
5     ('Poisson Reduit', pred_reduced, 'coral'),
6     ('Binomial Negatif', pred_nb, 'seagreen'),
7 ]
8 for ax, (name, pred, col) in zip(axes, pairs):
9     ax.scatter(y_test, pred, alpha=0.2, s=8, color=col)
10    lim = max(y_test.max(), pred.max())
11    ax.plot([0, lim], [0, lim], 'k--', lw=1.5)
12    ax.set_xlabel('Observe'); ax.set_ylabel('Predit')
13    ax.set_title(f'{name}\nMAE={mean_absolute_error(y_test,pred):.2f}
14                RMSE={rmse(y_test,pred):.2f}',
15                fontweight='bold')
16 plt.suptitle('Valeurs Observees vs Predites - Echantillon de Validation',
17             fontsize=13, fontweight='bold', y=1.02)
18 plt.tight_layout()
19 plt.savefig('fig6_obs_vs_pred.png', dpi=150, bbox_inches='tight')
20 plt.show()

```

Section 10 – Interprétation des coefficients par les IRR

Description : Cette dernière partie calcule les Incidence Rate Ratios, définis par $\exp(\hat{\beta})$, ainsi que leurs intervalles de confiance à 95%. Les IRR facilitent l'interprétation des coefficients du modèle de Poisson réduit en termes d'effet multiplicatif sur le nombre attendu de connexions suspectes.

Bloc 10.1

```

1 # Tableau des IRR (Incidence Rate Ratios) du modele reduit

```

```

2 coef_df = pd.DataFrame({
3     'beta (Coefficient)' : model_reduced.params,
4     'IRR = exp(beta)' : np.exp(model_reduced.params),
5     'p-valeur' : model_reduced.pvalues,
6     'IC 2.5%' : np.exp(model_reduced.conf_int()[0]),
7     'IC 97.5%' : np.exp(model_reduced.conf_int()[1])
8 })
9 coef_df['Significatif (p<0.05)'] = coef_df['p-valeur'] < 0.05
10
11 print("=" * 75)
12 print("TABLEAU DES IRR - MODLE POISSON RDUIT")
13 print("=" * 75)
14 print(coef_df.round(4).to_string())

```

Bloc 10.2

```

1 # Visualisation des IRR (hors Intercept)
2 sig = coef_df[coef_df['Significatif (p<0.05)']].drop('Intercept',
3     errors='ignore')
4
5 sig_sorted = sig.sort_values('IRR = exp(beta)', ascending=True)
6
7 plt.figure(figsize=(11, 7))
8 colors = ['steelblue' if v > 1 else 'coral' for v in sig_sorted['IRR =
9     exp(beta)']]
10 plt.barh(sig_sorted.index, sig_sorted['IRR = exp(beta)'], color=colors,
11     edgecolor='gray', alpha=0.85)
12 plt.axvline(1, color='black', linestyle='--', lw=1.5, label="IRR = 1 (pas
13     d'effet)")
14 plt.xlabel('IRR = exp(beta)', fontsize=13)
15 plt.title('Incidence Rate Ratios - Modele Poisson Reduit\n(variables
16     significatives, p < 0.05)',
17     fontweight='bold')
18 plt.legend(fontsize=11)
19 plt.tight_layout()

```

```
14 plt.savefig('fig7_irr.png', dpi=150, bbox_inches='tight')
15 plt.show()
```

Bloc 10.3

```
1  # Interpretation narrative des coefficients numeriques
2  vars_numeric = ['logged_in', 'serror_rate', 'rerror_rate',
3                  'same_srv_rate', 'diff_srv_rate',
4                  'dst_host_count', 'dst_host_srv_count']
5
6  print("=" * 70)
7  print("INTERPRETATION DES PRINCIPAUX COEFFICIENTS NUMERIQUES")
8  print("=" * 70)
9  for var in vars_numeric:
10     if var in coef_df.index:
11         irr = coef_df.loc[var, 'IRR = exp(beta)']
12         pval = coef_df.loc[var, 'p-valeur']
13         sig_marker = '[OK]' if pval < 0.05 else '[ERREUR]'
14         direction = 'augmente' if irr > 1 else 'diminue'
15         pct = abs((irr - 1) * 100)
16         print(f"\n{sig_marker} {var} - IRR = {irr:.4f} | p = {pval:.4f}")
17         if pval < 0.05:
18             print(f" -> Une augmentation d'1 unite {direction} le nombre")
19             print(f" de connexions suspectes attendu de {pct:.2f}%.")
```